Sprint 5 · Reto «¿La ocupación depende de esto?»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. Al dueño de TurisData le han dicho dos cosas: «*cuando hace calor se llena menos*» y «*los festivos llenan*». Antes de cambiar personal y precios, quiere saber **cuánto** cambia realmente la ocupación diaria con la temperatura y con los festivos, y **con qué seguridad** se puede afirmar. Os da la serie diaria de **tres años** (1096 días). Vuestra respuesta es un **cuaderno** con el análisis, una **base de datos** con los datos y una **respuesta al cliente de 150 palabras como máximo**.

Cómo se trabaja. Es el reto más «de números» hasta ahora, pero no hace falta ser matemático: os damos un **cuaderno plantilla** en Colab con las celdas por rellenar y unos **puntos de control** (`assert`) que os dicen al momento si lo que habéis escrito es correcto. Lo más importante: el **descenso del gradiente** (el método con el que un modelo «aprende» ajustando números poco a poco) lo escribís vosotros, sin usar la función ya hecha de una librería. En clase tenéis 10 minutos de planning (día 1) y, el día 2, la práctica de CRUD (que es el paso 1), **45 minutos de reto + 40 de entrega, review y retro**; lo que no quepa va a casa. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
El cuaderno plantilla `S05_07_reto_ocupacion`	Es el entregable: lo rellenáis celda a celda. Las celdas con `...` o `# TODO` son vuestras	Colab. Usad *Archivo → Guardar una copia en Drive* y compartidla con el equipo (permiso de edición)
Los datos `ocupacion_diaria.csv`	1096 días, **sintéticos**: `fecha` , `ocupacion_pct` , `temperatura_c` , `festivo` (1 = festivo) y `precio_medio`	El cuaderno los descarga solo con la función `dato("ocupacion_diaria.csv")`
Python y sus bibliotecas: `sqlite3` , `pandas` , `numpy` , `matplotlib` , `scipy` , `statsmodels`	Base de datos, tablas, gráficos, regresión y contrastes	Colab las trae instaladas; el cuaderno las importa en su celda 3
Cuadernos «en clase» del sprint que son la base del reto: `S05_03` (gradiente), `S05_04` (intervalos de confianza), `S05_05` (contrastes y `statsmodels`) y `S05_06` (CRUD, que es el paso 1)	Las pistas y el código parecido que podéis reutilizar	Colab. Su código explicado y el de `S05_07`
Presentaciones y apuntes	Repasar el gradiente (S05_03), la estadística (S05_04, S05_05) y el CRUD (S05_06)	S05_03 · S05_04 · S05_05 · S05_06 · apuntes
Cómo entregar en GitHub	Subir los ficheros a la carpeta `sprint-05/`	Guía de entrega

Recurso	Para qué lo usaréis	Dónde está
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender ideas y errores y pedir pistas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son seis pasos, de **105 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. En el horario del día 2 caben unos 70 minutos (la práctica de CRUD, que es el paso 1, y 45 de reto); los 35 que faltan se hacen en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30), y la subida puede hacerse hasta esa hora. Completad el nivel mínimo y dejad lo demás para casa. Todo se hace **dentro del cuaderno**, de arriba abajo; ejecutad las celdas en orden.

Paso 1 · Cargar los datos en una base de datos (día 2, 30 minutos)

1. Abrir el cuaderno `S05_07` en Colab, guardar una copia y compartirla con el equipo.
2. Ejecutar las dos primeras celdas de código (preparación e importaciones).
3. Crear `turisdata_reto.db` con la tabla `ocupacion_diaria` y un índice, y leerla con SQL.

En sencillo. Los datos del cliente vienen en un archivo de texto (`ocupacion_diaria.csv`): una fila por día, con la ocupación, la temperatura y si fue festivo. En este paso los metéis en una **base de datos**, que es como un armario con cajones ordenados: guarda tablas y deja pedir datos con preguntas muy precisas. Después hacéis una de esas preguntas (en un idioma que se llama **SQL**) para traer los datos de vuelta al cuaderno, ya con una columna nueva: el **mes**. No hace falta saber programar: escribís unas pocas líneas que aquí tenéis casi hechas, las ejecutáis y comprobáis que el cuaderno os da el visto bueno.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Celda	Cada caja del cuaderno. Las de texto se leen; las de código se **ejecutan** con el botón ▶ (o con `Mayús + Intro`).
Variable	Un nombre al que se guarda un valor para usarlo después. `datos = …` guarda lo que haya a la derecha con el nombre `datos`.
Librería	Un paquete de herramientas ya hechas. Se traen con `import`; `pd` es el apodo de `pandas`.
CSV	Un archivo de texto con una tabla: cada línea es una fila y los valores van separados por comas.
DataFrame	La tabla de `pandas` dentro de Python: filas, columnas con nombre y valores.
Base de datos y tabla	La base es el archivo (`turisdata_reto.db`); dentro hay tablas, como las hojas de un libro de cálculo.
SQL	El idioma para pedir datos a una base. `SELECT` significa «dame» y `FROM`, «de esta tabla».
Índice	Como el índice de un libro: hace que buscar por una columna (aquí, `fecha`) sea rápido.
Conexión (`con`)	La línea abierta con la base: por ella enviáis órdenes y recibís respuestas.

Cómo se hace

1. **Abrir vuestra copia del cuaderno.** Entrad en el cuaderno `S05_07` (se abre en Colab). Arriba, *Archivo → Guardar una copia en Drive*: así trabajáis en una copia vuestra y el original no se toca. Pulsad *Compartir* (arriba a la derecha) y añadid los correos del equipo con permiso de **Editor**. Una persona escribe y las demás miran y opinan; cambiad de turno cada pocos minutos.

2. **Ejecutar las dos celdas de preparación.** Son las dos primeras celdas con código («Preparación» e importaciones). Pulsad ▶ en cada una, **por orden**. No escriben nada en pantalla: sabréis que han terminado porque aparece un y un número entre corchetes (`[1]`, `[2]`). La primera crea la función `dato(…)`, que localiza (y descarga si hace falta) los datos del curso; la segunda trae las librerías.
3. **Ir a la celda del apartado «1 · Base de datos».** Veréis una línea que empieza por `# TODO` (es una pista, Python la ignora), tres puntos `...` y varias líneas con `assert` y `print`. **Borrad solo los tres puntos** y escribid ahí el código de abajo. No toquéis las líneas `assert`: son el **punto de control**, que comprueba vuestro trabajo.
4. **Escribir el código.** Copiad o escribid esto, respetando mayúsculas, comillas y paréntesis:

   ```
   datos = pd.read_csv(dato("ocupacion_diaria.csv"))
   con = sqlite3.connect(RUTA_BD)
   datos.to_sql("ocupacion_diaria", con, if_exists="replace", index=False)
   con.execute("CREATE INDEX IF NOT EXISTS idx_ocupacion_fecha ON ocupacion_diaria(fecha)")
   con.commit()
   diaria = pd.read_sql_query(
       "SELECT *, CAST(SUBSTR(fecha, 6, 2) AS INTEGER) AS mes FROM ocupacion_diaria", con)
   ```

   Qué hace cada línea:

   - `datos = pd.read_csv(dato("ocupacion_diaria.csv"))`: `dato(…)` da la ruta del archivo; `pd.read_csv` lo lee y lo guarda en la variable `datos` como un DataFrame (1096 filas: un día por fila, tres años).
   - `con = sqlite3.connect(RUTA_BD)`: abre (o crea) la base `turisdata_reto.db` y guarda la conexión en `con`. **Tiene que llamarse `con`**: el punto de control usa ese nombre.
   - `datos.to_sql(…)`: copia la tabla `datos` dentro de la base con el nombre `ocupacion_diaria`. `if_exists="replace"` borra la tabla vieja si ya existía, y así podéis ejecutar la celda varias veces sin duplicar filas; `index=False` evita una columna de números sobrante.
   - `con.execute("CREATE INDEX …")`: crea el índice. El nombre `idx_ocupacion_fecha` tiene que ser **exactamente ese**.
   - `con.commit()`: confirma los cambios, como «guardar». Sin esta línea la base puede quedarse sin ellos.
   - `diaria = pd.read_sql_query("SELECT …", con)`: lanza la pregunta SQL y guarda la respuesta en `diaria`. `SELECT *` pide todas las columnas; `SUBSTR(fecha, 6, 2)` recorta de `2023-07-14` los caracteres 6 y 7 (`07`, el mes); `CAST(… AS INTEGER)` lo convierte en el número `7`; y `AS mes` pone nombre a la columna nueva.

   Este paso es la práctica de CRUD (crear y leer datos en una base). Si ya hicisteis `S05_06`, podéis copiar el código de ahí.
5. **Ejecutar y mirar.** Pulsad ▶. Si todo va bien, abajo aparece `Punto de control 1 superado`. Para ver los datos, añadid una celda nueva con estas dos líneas (`diaria.shape` dice cuántas filas y columnas; `diaria.head()` muestra las cinco primeras filas):

   ```
   print(diaria.shape)
   diaria.head()
   ```

   Ojo: Colab solo enseña por sí mismo el resultado de la **última** línea de la celda. Por eso `diaria.shape` va dentro de `print(…)` y `diaria.head()` va al final. Si escribís las dos sin `print` en la misma celda, solo veréis una de ellas.

Qué deberíais ver. El mensaje `Punto de control 1 superado`. Con `diaria.head()`, una tabla de seis columnas (`fecha`, `ocupacion_pct`, `temperatura_c`, `festivo`, `precio_medio` y `mes`) cuya primera fila es `2023-01-01`, 89.0, 18.1, 1, 97.43 y mes 1; con `diaria.shape`, `(1096, 6)`. Y en el panel de archivos de Colab (el icono de la carpeta de la izquierda; si no aparece, pulsad el icono de refrescar) el fichero `turisdata_reto.db`. Si falla, el mensaje del `assert` dice qué falta (el archivo, las 1096 filas, la columna `mes` o el índice).

Hecho cuando: `turisdata_reto.db` existe, `diaria` tiene 1096 filas con la columna `mes` y el punto de control 1 está superado.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'pd' is not defined` (o `'sqlite3'`, `'dato'`, `'RUTA_BD'`)	No se ha ejecutado una celda de preparación, o Colab se reinició y se olvidó de todo.	Ejecutad las dos celdas de preparación por orden y después la vuestra.
`AssertionError: Debe existir el archivo turisdata_reto.db`	La celda corre, pero la base aún no se ha creado.	Comprobad que habéis borrado los `...` y que está la línea con `sqlite3.connect(RUTA_BD)`.
`AssertionError: Deben leerse 1096 días y has leído …`	`diaria` no tiene las 1096 filas.	Mirad `diaria.shape` y comprobad que leéis la tabla `ocupacion_diaria`.
`AssertionError: Falta la columna mes (1-12)`	La consulta SQL no crea la columna `mes`.	Revisad `CAST(SUBSTR(fecha, 6, 2) AS INTEGER) AS mes`: el 6, el 2 y el `AS mes`.
`AssertionError: Falta el índice idx_ocupacion_fecha`	El índice no existe o tiene otro nombre.	Copiad el nombre tal cual y no olvidéis `con.commit()`.
`FileNotFoundError: … 'ocupacion_diaria.csv'`	Habéis escrito el nombre del archivo directamente en `read_csv`.	Usad `pd.read_csv(dato("ocupacion_diaria.csv"))`: `dato(…)` sabe dónde está el archivo.
`ProgrammingError: Cannot operate on a closed database`	La última celda del cuaderno (`con.close()`) cierra la conexión y se ejecutó antes de tiempo.	Volved a ejecutar la celda del apartado 1 (abre otra conexión) y dejad `con.close()` para el final.

Paso 2 · Mirar los datos (día 2, 10 minutos)

1. Describir la serie y calcular la media de ocupación por mes.
2. Hacer al menos dos gráficos: ocupación por mes y ocupación frente a temperatura.
3. Escribir 2–3 observaciones con vuestras palabras.

En sencillo. Antes de calcular nada hay que **mirar los datos**, igual que se mira un mapa antes de salir de viaje. Este paso no tiene una respuesta «correcta»: sirve para descubrir qué hay (¿cuánto se llena el alojamiento de media?, ¿hay meses más llenos que otros?, ¿se ve alguna relación con el calor?) y para tener algo que contar. Escribís pocas líneas, las ejecutáis y **describís con vuestras palabras lo que veis**.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Media	La suma de todos los valores dividida entre cuántos son: el valor «típico».
Dispersión (`std`)	Cuánto se alejan los días de la media. Pequeña: los días se parecen; grande: hay días muy distintos.
Mínimo y máximo	El día de menor y el de mayor ocupación.
`groupby`	«Agrupar por»: reparte las filas en montones (uno por mes) y luego calcula algo de cada montón.
Diagrama de caja (*boxplot*)	Un dibujo por grupo. La caja contiene la mitad central de los días, la raya es la mediana (el valor del medio) y los palitos y puntos son los días más extremos.
Gráfico de dispersión (*scatter*)	Una nube de puntos: cada punto es un día, con la temperatura a lo ancho y la ocupación a lo alto.

Cómo se hace

1. **Describir la ocupación.** En la celda del apartado «2 · Primer vistazo a los datos» (borrad los `...`) escribid:

   ```
   print(diaria["ocupacion_pct"].describe())
   ```

   `diaria["ocupacion_pct"]` coge solo esa columna (entre corchetes y comillas, con el nombre exacto) y `.describe()` calcula de golpe la media, la dispersión, el mínimo y el máximo. **Los paréntesis son obligatorios**: sin ellos Python no calcula nada y escribe un texto raro que empieza por `<bound method`.

2. **Media por mes.** En la misma celda, debajo:

   ```
   medias_mes = diaria.groupby("mes")["ocupacion_pct"].mean()
   print(medias_mes.round(1))
   ```

   Se lee de izquierda a derecha: «coge `diaria`, agrúpala por `mes`, quédate con la columna `ocupacion_pct` y calcula la media de cada grupo». `.round(1)` deja un decimal. Para ver de un vistazo los meses más y menos llenos, probad `print(medias_mes.sort_values())` (ordena de menor a mayor).

3. **Primer gráfico: la ocupación mes a mes.**

   ```
   diaria.boxplot(column="ocupacion_pct", by="mes", figsize=(8, 4))
   plt.suptitle("")
   plt.title("Ocupación diaria según el mes")
   plt.xlabel("Mes (1 = enero, 12 = diciembre)")
   plt.ylabel("Ocupación (%)")
   plt.show()
   ```

   - `diaria.boxplot(…, by="mes")` dibuja una caja por cada mes; `figsize=(8, 4)` es el tamaño (ancho y alto).
   - `plt.suptitle("")` borra el título automático en inglés que pone la librería.
   - `plt.title(…)`, `plt.xlabel(…)` y `plt.ylabel(…)` ponen el título y los nombres de los ejes **en español y con unidades**. Es lo que pide el cuaderno.
   - `plt.show()` muestra el dibujo. Sin esta línea a veces solo sale un texto como `<Axes: …>`.

4. **Segundo gráfico: ocupación frente a temperatura.** En una celda nueva (botón **+ Código**):

   ```
   plt.figure(figsize=(6, 4))
   plt.scatter(diaria["temperatura_c"], diaria["ocupacion_pct"], s=8, alpha=0.5)
   plt.title("A más temperatura, menos ocupación")
   plt.xlabel("Temperatura (°C)")
   plt.ylabel("Ocupación (%)")
   plt.show()
   ```

   `plt.scatter(x, y)` dibuja un punto por día: primero lo que va a lo ancho (la temperatura) y después lo que va a lo alto (la ocupación). `s=8` hace los puntos pequeños y `alpha=0.5` los deja semitransparentes, para que se vea dónde se amontonan. Si dudáis del título, cambiadlo por lo que veáis vosotros.

5. **Escribir las observaciones.** Haced doble clic en la celda de texto «Mis observaciones» y sustituid los puntos por 2–3 frases con vuestras cifras. Pueden seguir estos moldes:

   - «En [meses] la ocupación media es [más alta / más baja]: alrededor del [cifra] %.»
   - «En el gráfico de puntos, a más temperatura [sube / baja] la ocupación.»
   - «Ojo: [el calor y ciertos meses van juntos], así que todavía no sabemos si el calor es la causa.»

   Es la base de vuestra respuesta al cliente, y no hace falta que sea técnica: escribid lo que le diríais a una persona que no ha visto los datos.

Qué deberíais ver.

- **El resumen** (`describe`): 1096 días, ocupación media ≈ 68,5 %, dispersión ≈ 9,6, mínimo 47,3 % y máximo 96,8 %.

- **Las medias por mes:** más altas en invierno (en torno al 78–79 % de media en enero, febrero y diciembre) y más bajas de mayo a octubre (en torno al 60–63 %).
- **El diagrama de caja:** cajas altas a los lados (invierno) y bajas en el centro (verano).
- **La nube de puntos: baja de izquierda a derecha**: a más temperatura, menos ocupación.

No hay `assert` en este paso: lo revisa el equipo.

Hecho cuando: hay dos gráficos con título y ejes en español y 2–3 observaciones escritas.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'diaria' is not defined`	La celda del apartado 1 no se ha ejecutado (o Colab se reinició).	Ejecutad desde la primera celda hacia abajo hasta la del apartado 1 y volved aquí.
`KeyError: 'ocupacion'` (con otro nombre)	El nombre de la columna no es exacto.	Escribid `diaria.columns` para ver los nombres: `fecha`, `ocupacion_pct`, `temperatura_c`, `festivo`, `precio_medio` y `mes`.
Sale `<bound method NDFrame.describe of …>` y muchas líneas	Falta el paréntesis: `describe` en vez de `describe()`.	Añadid `()` al final.
El gráfico sale con un título en inglés (*Boxplot grouped by mes*)	Falta `plt.suptitle("")`.	Añadid esa línea antes de `plt.title(…)`.
Sale un texto como `<Axes: …>` en vez del dibujo	Falta `plt.show()`.	Añadid `plt.show()` al final de la celda del gráfico.
Los dos gráficos salen uno encima del otro	Están en la misma celda sin `plt.figure(…)` entre ellos.	Poned cada gráfico en su celda, o empezad el segundo con `plt.figure(figsize=(6, 4))`.

Paso 3 · Regresión lineal desde cero (día 2, 35 minutos)

1. Escribir `gradiente_mse` y `entrenar` (el descenso del gradiente).
2. Ajustar el modelo simple (solo temperatura) y dibujar la curva de aprendizaje.
3. Ajustar el modelo con control (temperatura, festivo y mes).

En sencillo. El cliente quiere saber **cuánto cambia la ocupación por cada grado más de temperatura**. Si miráis la nube de puntos del paso anterior, entre ellos cabe una «mejor recta», y su **pendiente** responde justo a eso: cuántos puntos de ocupación sube o baja por cada °C. En vez de pedirle esa recta a una librería, la vais a encontrar vosotros con el **descenso del gradiente**. Es como bajar una montaña con niebla: no veis el valle, pero notáis hacia dónde baja el suelo bajo los pies, dais un pasito en esa dirección y repetís. Aquí la «montaña» es el **error** de la recta (cuánto se equivoca) y cada paso la corrige un poquito. Después repetís con más información (festivos y mes) para ver si el resultado cambia. Es el paso más largo y el más «de código», pero casi todo lo que tenéis que escribir está debajo, línea a línea.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Función (`def`)	Una receta con nombre: la escribís una vez y la usáis las veces que haga falta. `def entrenar(…):` define la receta `entrenar`.
Bucle (`for`)	Repetir las mismas líneas varias veces.
Lista	Una colección ordenada de valores. Con `.append(x)` se añade uno al final.
NumPy (`np`)	La librería de cálculo con números y tablas de números (*arrays*).

Palabra	Qué significa, en sencillo
`@`	El producto de matrices (el «multiplicar» de las tablas de números). `X.T` es la tabla de `X` girada (la traspuesta).
Error cuadrático medio (MSE)	La media de los errores al cuadrado: cuánto se aleja la recta de los puntos. Cuanto más pequeño, mejor.
Gradiente	La indicación de hacia dónde hay que mover la recta para que el error crezca más deprisa. Se va en sentido contrario.
Tasa de aprendizaje (`tasa`)	El tamaño del pasito. Muy pequeño: tarda mucho. Muy grande: se pasa de largo y el error se dispara.
Época	Una vuelta completa del bucle (calcular el gradiente y dar un paso).
Estandarizar	Restar la media y dividir entre la desviación, para que las variables tengan una escala parecida y el descenso no se desboque.
Pesos (`w`)	Los números que definen la recta: la constante y la pendiente.
Variable indicadora	Una columna de 0 y 1 (por ejemplo, «¿es julio?: sí o no»). Así se meten los meses en la cuenta.

Cómo se hace

1. **Saber qué llega a las funciones.** `y` es lo que queremos predecir (la ocupación). `X` es una tabla de números con una **columna de unos** (para la constante de la recta) y las variables explicativas. `w` son los pesos: uno por columna de `X`. Con `X @ w` se calcula la predicción de todos los días de una vez.
2. **Escribir las dos funciones.** En la celda del apartado «3 · Regresión lineal desde cero» (borrad los `...`). **Importante:** los `assert` y el `print` del final de la celda se quedan **al final**; todo el código de este paso (funciones, modelo simple y modelo con control) va **entre la línea `# TODO` y los `assert`**, nunca debajo de ellos, porque los `assert` comprueban lo que se ha calculado antes:

   ```
   def gradiente_mse(X, y, w):
       return 2 / len(y) * X.T @ (X @ w - y)

   def entrenar(X, y, tasa, epocas):
       w = np.zeros(X.shape[1])
       historial = []
       for _ in range(epocas):
           w = w - tasa * gradiente_mse(X, y, w)
           historial.append(np.mean((X @ w - y) ** 2))
       return w, historial
   ```

   Qué hace cada parte:

   - `gradiente_mse(X, y, w)` calcula el gradiente: `X @ w - y` es el error de cada día, y `2 / len(y) * X.T @ (…)` los resume en un número por peso. Basta con teclearla bien; la idea es la del cuaderno `S05_03`.
   - `w = np.zeros(X.shape[1])`: empieza con todos los pesos a cero (una recta plana). `X.shape[1]` es el número de columnas de `X`.
   - `historial = []`: una lista vacía donde se apuntará el error de cada vuelta.
   - `for _ in range(epocas):`: repite lo de debajo `epocas` veces. Lo que está **hacia dentro** (con sangría) es lo que se repite.
   - `w = w - tasa * gradiente_mse(X, y, w)`: el pasito: se mueven los pesos en sentido contrario al gradiente, en la medida que marca `tasa`.
   - `historial.append(np.mean((X @ w - y) ** 2))`: calcula el error (MSE) actual y lo apunta en la lista.
   - `return w, historial`: devuelve los pesos finales y la lista de errores. **Sin `scikit-learn`**: el reto pide hacerlo a mano.

3. **Modelo simple (solo temperatura).** En la misma celda, debajo de las funciones y **encima de los `assert`**:

```
y = diaria["ocupacion_pct"].to_numpy()
t = diaria["temperatura_c"]
t_est = (t - t.mean()) / t.std()
X = np.column_stack([np.ones(len(diaria)), t_est.to_numpy()])
w, historial = entrenar(X, y, 0.1, 2000)
pendiente_simple = w[1] / t.std()
```

   - `y = …to_numpy()`: la ocupación como tabla de números de NumPy.
   - `t_est = (t - t.mean()) / t.std()`: la temperatura **estandarizada** (restada la media y dividida entre la desviación).
   - `np.column_stack([…])`: pega columnas en una tabla; la primera es una columna de unos (`np.ones`), la segunda, la temperatura estandarizada.
   - `entrenar(X, y, 0.1, 2000)`: hace 2000 vueltas con pasitos de tamaño 0,1.
   - `w[1] / t.std()`: `w[1]` es la pendiente **en «desviaciones de temperatura»**; al dividir entre `t.std()` (unos 3,2 °C) se **deshace la estandarización** y queda en **puntos de ocupación por cada °C**, que es lo que entiende el cliente. Esa es `pendiente_simple`.

4. **Curva de aprendizaje.** En la celda siguiente del cuaderno (la del `plt.plot`):

```
plt.plot(historial)
plt.title("Curva de aprendizaje: el error baja y se estabiliza")
plt.xlabel("Época (vuelta del descenso)")
plt.ylabel("Error cuadrático medio (MSE)")
plt.show()
```

   Debajo, una línea de texto: ¿baja y se aplana? (entonces el modelo ha **convergido**: ya no mejora). Para ver mejor la bajada del principio, probad `plt.plot(historial[:100])`.

5. **Modelo con control: temperatura, festivo y mes.** «Controlar» significa meter también el festivo y el mes, para ver qué pasa con la temperatura cuando ya se tiene en cuenta la época del año. En la misma celda del apartado 3, debajo de lo anterior y **todavía encima de los `assert`**:

```
meses = pd.get_dummies(diaria["mes"], drop_first=True, dtype=float)
M = pd.concat([diaria[["temperatura_c", "festivo"]].astype(float), meses], axis=1)
medias, desv = M.mean(), M.std()
Xc = np.column_stack([np.ones(len(M)), ((M - medias) / desv).to_numpy()])
wc, historial_c = entrenar(Xc, y, 0.1, 2000)
efecto_temp_control = wc[1] / desv["temperatura_c"]
efecto_festivo_control = wc[2] / desv["festivo"]
```

   - `pd.get_dummies(diaria["mes"], drop_first=True, …)`: convierte el mes en **11 columnas** de 0 y 1 (una por mes menos uno: el mes que falta se deduce de los demás, y así no se repite información).
   - `pd.concat([…], axis=1)`: pega a la derecha la temperatura, el festivo y las 11 columnas de meses en una sola tabla `M` (13 columnas).
   - `medias, desv = M.mean(), M.std()`: la media y la desviación de **cada** columna. Se guardan porque hace falta `desv` al final.
   - `Xc = …((M - medias) / desv)…`: estandariza todas las columnas y les pega delante la columna de unos.
   - `wc[1]` es el peso de la temperatura y `wc[2]`, el del festivo (el orden de las columnas: unos, temperatura, festivo, meses). Cada uno se divide entre **la desviación de su propia variable** para volver a las unidades originales: puntos por °C para la temperatura y puntos por día festivo (la desviación del festivo es la suya, no la de la temperatura).

6. **Ejecutar y comprobar.** Pulsad ▶. Los `assert` de debajo comparan vuestros tres números con los esperados.

Qué deberíais ver.

- **El error (MSE)** arranca en unos 3081 y baja a ≈ 56 en menos de 50 vueltas; el resto de las 2000 vueltas casi no cambia nada. Por eso la curva cae en picado y **se aplana enseguida**.

- `pendiente_simple` **≈ −1,86**: por cada grado más, la ocupación baja unos 1,86 puntos.
- El mensaje `Punto de control 2 superado: simple -1.86 · con control: temp -0.09, festivo +9.11`.
- Fijaos en que **la temperatura casi deja de importar** (−0,09) cuando se controla el mes, y que el festivo suma unos 9 puntos.

En el nivel mínimo, el punto de control dará error en las líneas del modelo con control hasta que lo hagáis.

Hecho cuando: `pendiente_simple` sale ≈ −1,86, la curva de aprendizaje está dibujada y comentada y (en el estándar) el punto de control 2 está superado.

Si algo falla

Veis...	Qué significa	Qué hacer
`NameError: name 'np' is not defined` (o `'diaria'`)	No se han ejecutado la celda de importaciones o la del apartado 1.	Ejecutad desde la primera celda hacia abajo.
El error crece sin parar, sale `nan` o `inf`, o números enormes	La tasa es demasiado grande (o no habéis estandarizado). Con la temperatura sin estandarizar y tasa 0,1 el error llega a cifras de más de 100 cifras.	Estandarizad la temperatura y usad tasa 0,1; si aun así se dispara, bajadla (0,05 o 0,01).
`AssertionError: La pendiente simple debería ser ≈ -1.86 puntos/°C y tienes -6.037`	Habéis dejado la pendiente «en desviaciones»: falta deshacer la estandarización.	Dividid `w[1]` entre `t.std()`: `pendiente_simple = w[1] / t.std()`.
`AssertionError` en el efecto con control (temperatura cerca de −1,86 o festivo raro)	Falta algún mes en el modelo, o se ha dividido entre la desviación equivocada.	Comprobad que `M` tiene 13 columnas (`M.shape`) y que cada coeficiente se divide entre **su** desviación.
`NameError: name 'pendiente_simple' is not defined` (o `efecto_temp_control`, `efecto_festivo_control`)	Ese código está **debajo** de los `assert` (se ejecuta después de comprobarlo), falta en la celda, o el nombre no coincide con el del cuaderno.	Poned todo el código **encima** de los `assert` y usad exactamente esos tres nombres.
`ValueError` que habla de `shapes` o `matmul`	Los tamaños no cuadran: `X` y `w` deben tener tantas columnas como pesos.	Mirad `X.shape` y `w.shape`: el segundo número de `X` debe ser el tamaño de `w`.
La curva es una raya casi vertical y plana	Es normal: el error baja casi todo en las primeras vueltas.	Dibujad solo el principio: `plt.plot(historial[:100])`.

Paso 4 · Comparar con una librería (día 2, 10 minutos)

1. Ajustar los mismos dos modelos con `statsmodels`.
2. Reunir los tres coeficientes en la tabla `comparacion`.
3. Comprobar que coinciden con los vuestros.

En sencillo. ¿Cómo sabéis que vuestro descenso del gradiente funciona? Comparándolo con una **librería profesional** que resuelve el mismo problema. `statsmodels` ajusta la recta con otro método (la fórmula clásica, mínimos cuadrados ordinarios) y, además, trae de serie las cifras estadísticas que necesitaréis en el paso siguiente. Si vuestro código y la librería dan **los mismos números**, vuestro código está bien; si no, hay algo que revisar. Es, además, el momento de ver qué se gana usando una librería: dos líneas hacen lo que vosotros habéis programado a mano.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
`statsmodels` (`smf`)	Librería de estadística. `smf` es el apodo que le pone el cuaderno al módulo de modelos con fórmulas.

Palabra	Qué significa, en sencillo
OLS (mínimos cuadrados)	El método clásico para ajustar una recta: busca los pesos que dejan el error cuadrático lo más pequeño posible (lo mismo que persigue vuestro descenso).
Fórmula (`"y ~ x1 + x2"`)	Una frase que describe el modelo: la `~` se lee «se explica por». `ocupacion_pct ~ temperatura_c` es «la ocupación se explica por la temperatura».
`C(mes)`	«Trata `mes` como categorías, no como un número». La librería crea sola las 11 columnas de 0 y 1 que vosotros hicisteis con `get_dummies` .
`.fit()`	«Ajusta»: calcula los pesos. Devuelve un objeto con todos los resultados.
`.params`	Los pesos (coeficientes) del modelo ajustado, con su nombre.

Cómo se hace

1. **Ajustar los dos modelos.** En la celda del apartado «4 · Contraste con una librería» (borrad los `...`):

```
ols_simple = smf.ols("ocupacion_pct ~ temperatura_c", data=diaria).fit()
ols_control = smf.ols("ocupacion_pct ~ temperatura_c + festivo + C(mes)", data=diaria).fit()
```

   - `smf.ols("…", data=diaria)` prepara el modelo: la fórmula dice qué se explica y con qué, y `data=diaria` indica de qué tabla salen las columnas.
   - `.fit()` lo ajusta. Guardad el resultado en `ols_simple` y `ols_control` (con esos nombres: se usan en el paso 5).

2. **Montar la tabla `comparacion` .** Debajo:

```
comparacion = pd.DataFrame(
    {"desde_cero": [pendiente_simple, efecto_temp_control, efecto_festivo_control],
     "statsmodels": [ols_simple.params["temperatura_c"],
                     ols_control.params["temperatura_c"],
                     ols_control.params["festivo"]]},
    index=["pendiente_simple", "efecto_temp_control", "efecto_festivo_control"])
```

   La tabla tiene **dos columnas** (`desde_cero` : vuestros números; `statsmodels` : los de la librería) y **tres filas** (las que dice `index=`). `ols_control.params["festivo"]` es el peso de la columna `festivo` , y así con los demás.

3. **Ejecutar.** Pulsad ▶: el cuaderno comprueba que la tabla es de 3 × 2 y que las dos columnas coinciden (se admite una diferencia de 0,02), y luego imprime la tabla.

Qué deberíais ver. `Punto de control 3 superado` y la tabla con dos columnas casi idénticas:

```
                     desde_cero  statsmodels
pendiente_simple         -1.860       -1.860
efecto_temp_control      -0.089       -0.089
efecto_festivo_control    9.111        9.111
```

Para curiosear, `print(ols_control.summary())` muestra el informe completo de la librería: lo usaréis en el paso 5.

Hecho cuando: `comparacion` tiene 3 × 2 valores que coinciden y el punto de control 3 está superado.

Si algo falla

Veis...	Qué significa	Qué hacer
`AssertionError: Tu implementación no coincide con statsmodels:` y la tabla	Vuestros números y los de la librería se separan más de 0,02.	Mirad en qué fila: si es la primera, repasad el modelo simple (¿dividisteis entre `t.std()` ?); si son las otras dos, el modelo con control. Antes de buscar un fallo, probad con más épocas (por ejemplo, 5000).

Veis…	Qué significa	Qué hacer
`AssertionError: La tabla debe tener 3 filas y 2 columnas`	Falta una fila o una columna, o hay alguna de más.	Comprobad `comparacion.shape`: debe ser `(3, 2)`.
`NameError: name 'pendiente_simple' is not defined` (o `efecto_…`)	No se ejecutó la celda del apartado 3 (o Colab se reinició).	Ejecutad de nuevo desde arriba, o al menos hasta el apartado 3.
`KeyError: 'temperatura_c'` en `.params[…]`	Nombre mal escrito dentro de los corchetes.	Mirad los nombres con `ols_control.params.index` (verás `Intercept`, los meses `C(mes)[T.2]`, …, `temperatura_c` y `festivo`).
`PatsyError: Error evaluating factor: NameError: name '…' is not defined`	La fórmula tiene un nombre de columna que no existe o un error de escritura.	Revisad la fórmula: `ocupacion_pct`, `temperatura_c`, `festivo` y `C(mes)` tal cual.

Paso 5 · Medir la seguridad del resultado (día 2, 10 minutos)

1. Obtener el p-valor, el intervalo de confianza y el R^2.
2. Calcular el tamaño del efecto.
3. Explicar cada cifra con palabras.

En sencillo. Ya tenéis números (una pendiente, un efecto del festivo…), pero el cliente querrá saber **cuánto se puede fiar de ellos**. Con solo tres años de datos, los resultados podrían ser casualidad. La estadística tiene herramientas para medir esa incertidumbre, y en este paso sacáis cuatro cifras del modelo de `statsmodels`: un **p-valor** (¿podría ser casualidad?), un **intervalo de confianza** (entre qué valores está el efecto) y el **R^2** (cuánto explica el modelo). Después las **traducís a palabras**: es la parte que más cuenta, porque la respuesta al cliente se escribe con esas frases.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
p-valor	«Si en realidad no hubiera ningún efecto, ¿qué probabilidad había de ver unos datos tan llamativos como estos?». Si es muy pequeño (menos de 0,05), sería raro que fuera casualidad. Si es grande, **los datos no bastan para afirmar un efecto** (no es lo mismo que «no hay efecto»).
Intervalo de confianza del 95 %	Un tramo de valores (de A a B) en el que, con bastante seguridad, está el efecto real. Cuanto más estrecho, más preciso.
R^2	Cuánta de la variación de la ocupación explica el modelo: 0 es «nada» y 1 es «todo».
Tamaño del efecto	Cuánto cambia el resultado en las unidades de siempre (puntos de ocupación). Un p-valor pequeño dice que hay efecto, pero no si es grande o pequeño.
t de Welch	Una prueba que compara las medias de dos grupos (aquí, días festivos y no festivos) y devuelve un p-valor.
`.loc[…]`	Para elegir filas (y columnas) de una tabla por una condición o por su nombre.
`1e-30`	Notación científica: el 1 está en el lugar 30 después de la coma (0,000…001). Es un número minúsculo.

Cómo se hace

1. **Separar los días festivos de los demás.** En la celda del apartado «5 · Responder con estadística» (borrad los `...`):

   ```
   festivos = diaria.loc[diaria["festivo"] == 1, "ocupacion_pct"]
   no_festivos = diaria.loc[diaria["festivo"] == 0, "ocupacion_pct"]
   ```

   `diaria.loc[condición, "ocupacion_pct"]` se queda con la columna de ocupación **solo en las filas que cumplen la condición**. `==` significa «es igual a» (con un solo `=` se guardaría un valor, no se compararía).

2. **Sacar las cuatro cifras.**

   ```
   p_temp_control = ols_control.pvalues["temperatura_c"]
   ic_festivo = tuple(ols_control.conf_int().loc["festivo"])
   p_festivo_ttest = stats.ttest_ind(festivos, no_festivos, equal_var=False).pvalue
   r2_control = ols_control.rsquared
   ```

   - `ols_control.pvalues["temperatura_c"]`: el p-valor de la temperatura **en el modelo con control** (el que ya tiene el mes).
   - `ols_control.conf_int().loc["festivo"]`: la fila del festivo de la tabla de intervalos de confianza (95 %); `tuple(…)` la guarda como pareja de dos números (A, B).
   - `stats.ttest_ind(…, equal_var=False).pvalue`: el t de Welch entre festivos y no festivos; `equal_var=False` es lo que lo convierte en la versión de Welch, y `.pvalue` se queda solo con el p-valor.
   - `ols_control.rsquared`: el R^2 del modelo con control.

3. **Mirar las cifras.** Para verlas, en una celda nueva:

   ```
   print(round(p_temp_control, 3), tuple(round(x, 2) for x in ic_festivo), p_festivo_ttest, round(r2_control, 3))
   ```

4. **Calcular el tamaño del efecto.** No hay que calcular nada nuevo: ya lo tenéis en los coeficientes. Con un grado más, la ocupación cambia en `efecto_temp_control` puntos; un día festivo, en `efecto_festivo_control` puntos. Para ver el «antes y después de controlar», comparad las medias a pelo: `festivos.mean()` y `no_festivos.mean()`.

5. **Explicar cada cifra con palabras.** En una celda de texto, una frase por cifra, con estos moldes:

   - Festivo: *«Con un día festivo, la ocupación sube unos [X] puntos (con un 95 % de confianza, entre [A] y [B])»*.
   - Temperatura: *«Una vez descontado el mes, la temperatura casi no cambia la ocupación: [X] puntos por grado, con un p-valor de [p]: con estos datos no podemos afirmar que influya»*.
   - p-valor: *«Si no hubiera ningún efecto, sería muy raro (o no) ver unos datos así»*: usad «muy raro» si el p-valor es pequeño y «nada raro» si es grande.
   - R^2: *«El modelo explica un [X] % de la variación de la ocupación; el resto depende de otras cosas»*.

   Escribid las frases **con vuestras cifras**, porque son la base de la respuesta al cliente.

Qué deberíais ver. `Punto de control 4 superado` y, con el `print` de arriba, `0.481 (7.35, 10.87) 1.127026104033321e-30 0.579`:

- `p_temp_control` ≈ 0,48: la temperatura **no** es significativa una vez controlado el mes.
- `ic_festivo` ≈ (7,35 ; 10,87): un festivo suma entre 7 y 11 puntos.
- `p_festivo_ttest` minúsculo (del orden de `1e-30`): sin duda, los festivos se llenan más.
- `r2_control` ≈ 0,58: el modelo explica casi el 58 % de la variación.

Un detalle interesante: sin controlar nada, los días festivos tienen de media un 83,9 % de ocupación y los demás un 67,6 % (unos 16 puntos de diferencia); controlando el mes, el efecto del festivo baja a unos 9 puntos. Parte de la diferencia viene de que los festivos caen en meses de ocupación alta.

Hecho cuando: el punto de control 4 está superado y cada cifra tiene su frase explicada con palabras.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'ols_control' is not defined`	No se ha ejecutado el paso 4 (o Colab se reinició).	Ejecutad desde arriba, o al menos hasta el apartado 4.
`KeyError: 'festivo'` en `.loc[…]`	El nombre no coincide con una fila de la tabla.	Escribid `ols_control.conf_int()` para ver los nombres de las filas.
`AssertionError: Con control de mes, la temperatura no es significativa (p ≈ 0.48) y tienes p = …`	Habéis cogido el p-valor del modelo simple (`ols_simple`) en lugar del del modelo con control.	Usad `ols_control`.
`AssertionError: El IC del festivo debería ser ≈ (7.35, 10.87) y tienes …`	Habéis cogido otra fila o otro modelo.	Usad `ols_control.conf_int().loc["festivo"]`.
`AssertionError: El t-test de festivos debe dar un p-valor minúsculo`	Habéis comparado grupos mal formados (por ejemplo, con `=` en lugar de `==`).	Comprobad que `festivos.shape` es `(61,)` y `no_festivos.shape` es `(1035,)`.
El p-valor sale como `1.127026104033321e-30` y no entendéis qué es	Es notación científica: un número minúsculo.	Leedlo como «prácticamente cero»; en la explicación basta con decir «muy pequeño (menos de 0,001)».

Paso 6 · Responder al cliente en 150 palabras, reiniciar y revisar (día 2, 10 minutos)

1. Escribir la respuesta al cliente y contar las palabras.
2. Reiniciar el cuaderno y ejecutarlo todo de arriba abajo.
3. Pedir a alguien que no escribió esa parte que lo revise con la rúbrica y anotar el resultado.

En sencillo. Es el paso de **cerrar el trabajo**. Primero, contestar al cliente: una persona del alojamiento que no sabe de estadística necesita leer en 150 palabras qué habéis encontrado, qué le recomendáis y de qué no os fiáis. Después, comprobar que el cuaderno **funciona de principio a fin** (si alguien lo abre mañana y lo ejecuta, sale lo mismo) y que otra persona del equipo, con ojos frescos, lo revisa. No hay código nuevo, salvo una línea para contar palabras.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Reiniciar y ejecutar todo	Borra la memoria del cuaderno y ejecuta todas las celdas de arriba abajo. Es la prueba de que no depende de nada «escondido».
Reproducibilidad	Que cualquiera, ejecutando el cuaderno, obtenga los mismos resultados.
Rúbrica	La tabla con la que se evalúa el reto: cuatro criterios, cada uno de 1 a 4.
Comillas triples (`"""…"""`)	Sirven para guardar un texto de varias líneas en una variable.
`len` y `.split()`	`.split()` corta un texto en palabras y `len(…)` cuenta cuántas hay.

Cómo se hace

1. **Escribir la respuesta.** En la celda de texto de la sección «6 · Respuesta al cliente» (doble clic para editarla). Contestad **cuatro cosas**, para una persona que no es técnica:
    1. **Qué encontráis**, con cifras e incertidumbre, para la temperatura y para los festivos. *Molde:* «Con los datos de tres años, por cada grado más de temperatura la ocupación [sube/baja] [X] puntos (entre [A] y [B]), una vez descontado el mes. Los festivos suman unos [X] puntos…».
    2. **Qué recomendáis**, algo que el cliente pueda hacer con el personal o los precios. *Molde:* «Recomendamos…».
    3. **Qué no podéis afirmar**: correlación no es causalidad, el mes confunde, los datos son sintéticos y de un solo alojamiento. *Molde:* «No podemos afirmar que… porque…».
    4. **Una frase de IA responsable**, por ejemplo que el modelo no debería usarse para fijar precios sin revisión humana.

    Toda cifra sale de vuestro cuaderno: nada de números de memoria ni de lo que diga un asistente de IA.
2. **Contar las palabras.** Pegad la respuesta dentro de las comillas de esta celda de código y ejecutadla:

    ```
    texto = """Pegad aquí vuestra respuesta al cliente."""
    print(len(texto.split()), "palabras")
    ```

    Si sale más de 150, **recortad**: quitad adornos y dejad cifras, recomendación y límites.
3. **Reiniciar y ejecutar todo.** Arriba, *Entorno de ejecución → Reiniciar y ejecutar todo* (y confirmad). Tardará un poco. Mirad **hasta el final** que ninguna celda muestre un error en rojo. Si alguna falla, corregidla **antes de entregar** y repetid. Marcad también la lista de la sección 7 del cuaderno.
4. **Dos celdas de texto al final del cuaderno.**
    - **«Uso de IA»:** qué le pedisteis a la IA, qué comprobasteis y qué cambiasteis. Si no la usasteis, decidlo. *Molde:* «Le pedí [qué]; comprobé [cómo]; cambié [qué]».
    - **«Revisión con la rúbrica»:** copiad y rellenad esto, siguiendo los cinco pasos de «Cómo se revisa» de entrega y rúbrica:

```
## Revisión con la rúbrica
**Revisado por:** (nombre) · **Fecha:** (dd/mm/aaaa)

| Criterio | Nivel (1-4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** (una cosa concreta)
```

```
Si no cabe en los 10 minutos, la revisión se hace al empezar la entrega.
```

5. **La revisión.** La hace una persona que **no escribió** esa parte del cuaderno: lee el cuaderno ya reiniciado, pone un nivel de 1 a 4 en cada criterio y apunta **una frase de prueba** (lo que ha comprobado: «he reiniciado y todo corre»). Después, una mejora concreta para la próxima vez.

Qué deberíais ver. El cuaderno entero ejecutado sin errores y con los cuatro puntos de control superados, una respuesta de 150 palabras o menos y las dos celdas finales rellenas.

Hecho cuando: *Reiniciar y ejecutar todo* funciona sin errores, la respuesta cumple las cuatro preguntas y la «Revisión con la rúbrica» tiene un nivel y una frase de prueba por criterio.

Después, 40 minutos de entrega, review y retro (comparten tiempo con el Sprint 4):

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) en Colab, *Archivo* → *Descargar* → *Descargar .ipynb* (después de *Reiniciar y ejecutar todo*) y descargad también `turisdata_reto.db` desde el panel de archivos (clic derecho sobre el archivo → *Descargar*); (2) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-05`**; (3) *Add file* → *Upload files* y arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace: repartid las subidas entre personas (por ejemplo, una sube el cuaderno y otra la base de datos). Nombres de archivo sin espacios ni tildes. El CSV va a `data/` solo si aún no está.
- **La autoevaluación.** Cada persona rellena y sube la suya a `sprint-05/`; la coevaluación va dentro del mismo fichero.
- **La review (5 minutos por equipo).** Enseñad: (1) la pregunta y la base de datos; (2) cómo converge vuestro descenso del gradiente y su comparación con la librería; (3) el resultado clave con su intervalo; (4) cómo habéis distinguido lo que los datos muestran de lo que no pueden demostrar; (5) la recomendación al cliente.

Qué deberíais ver. En la carpeta `sprint-05` del repositorio, el cuaderno, la base de datos y una hoja de autoevaluación por persona (**2 + N**), con *commits* de personas distintas en el historial (el icono del reloj).

Si algo falla

Veis...	Qué significa	Qué hacer
Al *Reiniciar y ejecutar todo* sale un error en una celda y se para	Una celda depende de otra que no está arriba o que falla.	Leed el error de la **primera** celda en rojo; corregidla y repetid.
`NameError` justo después de reiniciar	Algo se definió en una celda que ahora está debajo de donde se usa.	Mover la celda arriba, o ejecutar siempre de arriba abajo.
`ProgrammingError: Cannot operate on a closed database`	Se ejecutó `con.close()` antes de tiempo.	Ejecutad todo desde arriba con *Reiniciar y ejecutar todo*.
El texto da más de 150 palabras	La respuesta es demasiado larga.	Recortad: quedaos con cifra e intervalo, una recomendación, los límites y la frase de IA responsable.
No sabéis qué nivel poner en la rúbrica	Los niveles están descritos en entrega y rúbrica.	Leed la fila del criterio y elegid el nivel que describa lo que habéis comprobado; escribid la prueba.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código y explicaciones que parecen correctos y no lo son**. Reglas:

- **El descenso del gradiente lo escribís vosotros.** Si se lo pedís a la IA, no podréis explicarlo en la review.
- **Cualquier cifra** que aparezca en la respuesta tiene que salir de vuestro cuaderno, no de la IA.
- **Declarad** en la celda «Uso de IA» qué pedisteis y qué comprobasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender una idea	«Explícame el descenso del gradiente como si fuera bajar una montaña con niebla, sin fórmulas»	Que podéis explicarlo sin mirar
Entender un error de código	«Mi curva de aprendizaje sube en vez de bajar. Este es mi bucle: *[código]*. ¿Qué puede pasar?»	Que probáis la sugerencia y la curva baja
Interpretar un resultado	«Tengo un p-valor de 0,03 y un intervalo de confianza de [−1,1; −0,4]. Explícamelo para un directivo, con un ejemplo»	Que la explicación coincide con **vuestros** números
Revisar la redacción	«Revisa mi respuesta de 150 palabras: ¿hay jerga o algo ambiguo? No cambies las cifras»	Que las cifras siguen siendo las vuestras

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
La curva de aprendizaje sube o da `NaN`	Tasa de aprendizaje demasiado grande o variables sin estandarizar	Reducir la tasa y estandarizar la temperatura
Vuestra recta no coincide con `statsmodels`	Pocas épocas o tasa muy pequeña	Más épocas o ajustar la tasa
El punto de control 1 falla por el índice o por `con`	El índice no se llama `idx_ocupacion_fecha` o la conexión no se llama `con`	Usar exactamente esos nombres
Error al repetir la celda de la base	Falta `if_exists="replace"` al guardar la tabla	Añadirlo
Decir que «el calor provoca…»	Se confunde correlación con causa	Hablar de «se asocia con» y explicar los límites
Olvidar la estacionalidad	El verano coincide con calor Y con vacaciones	Incluir `mes` en el modelo con control
Dar solo el p-valor	«Es lo que importa»	Dar también el **tamaño** del efecto y el **intervalo**
El cuaderno solo funciona en un orden raro	Se ejecutaron celdas sueltas	*Reiniciar y ejecutar todo* antes de entregar
La respuesta pasa de 150 palabras	No se ha contado	`len(texto.split())` y recortar

Ayuda 3 · Si os atascáis

1. Leed el mensaje del `assert` que falla: dice qué esperaba y qué recibió. Probad la función con un ejemplo pequeño en una celda aparte.
2. Repasad el cuaderno de esa parte: `S05_03` para el gradiente, `S05_05` para `statsmodels` y los p-valores, `S05_06` para la base de datos; y el código explicado.
3. Preguntad al equipo y, después, al docente (no más de 15 minutos atascados).
4. Cuando hayáis terminado el paso o agotado las pistas, contrastad con el solucionario del reto: el código completo de cada paso, lo que tiene que salir, cómo se lee cada cifra y una respuesta de ejemplo. Después de mirarlo, cerradlo y volved a escribirlo sin mirar.